

DECORATOR PATTERN

Structural Design Pattern trong Order Management

1 Mục tiêu bài học

Sau tuần này, sinh viên có thể:

- Hiểu bản chất "gói" (wrapping) đối tượng
- Giải quyết vấn đề bùng nổ lớp con (Class Explosion)
- Áp dụng Decorator vào hệ thống Order:
 - Tính thuế (Tax)
 - Phí vận chuyển (Shipping)
 - Giảm giá (Discount)
- Hiểu cơ chế Middleware trong ASP.NET Core

2 Đặt vấn đề: Class Explosion

✗ Bài toán: Cần tính toán giá Order với nhiều tùy chọn.

- Nếu dùng kế thừa (Inheritance):
 - Order
 - OrderWithTax
 - OrderWithShipping
 - OrderWithTaxAndShipping
 - OrderWithTaxAndDiscount
 - OrderWithTaxAndShippingAndDiscount...

⚠ Hậu quả: Class Explosion (Bùng nổ số lượng lớp).
Khó bảo trì, code lặp lại nhiều.

3 Giải pháp: Decorator Pattern

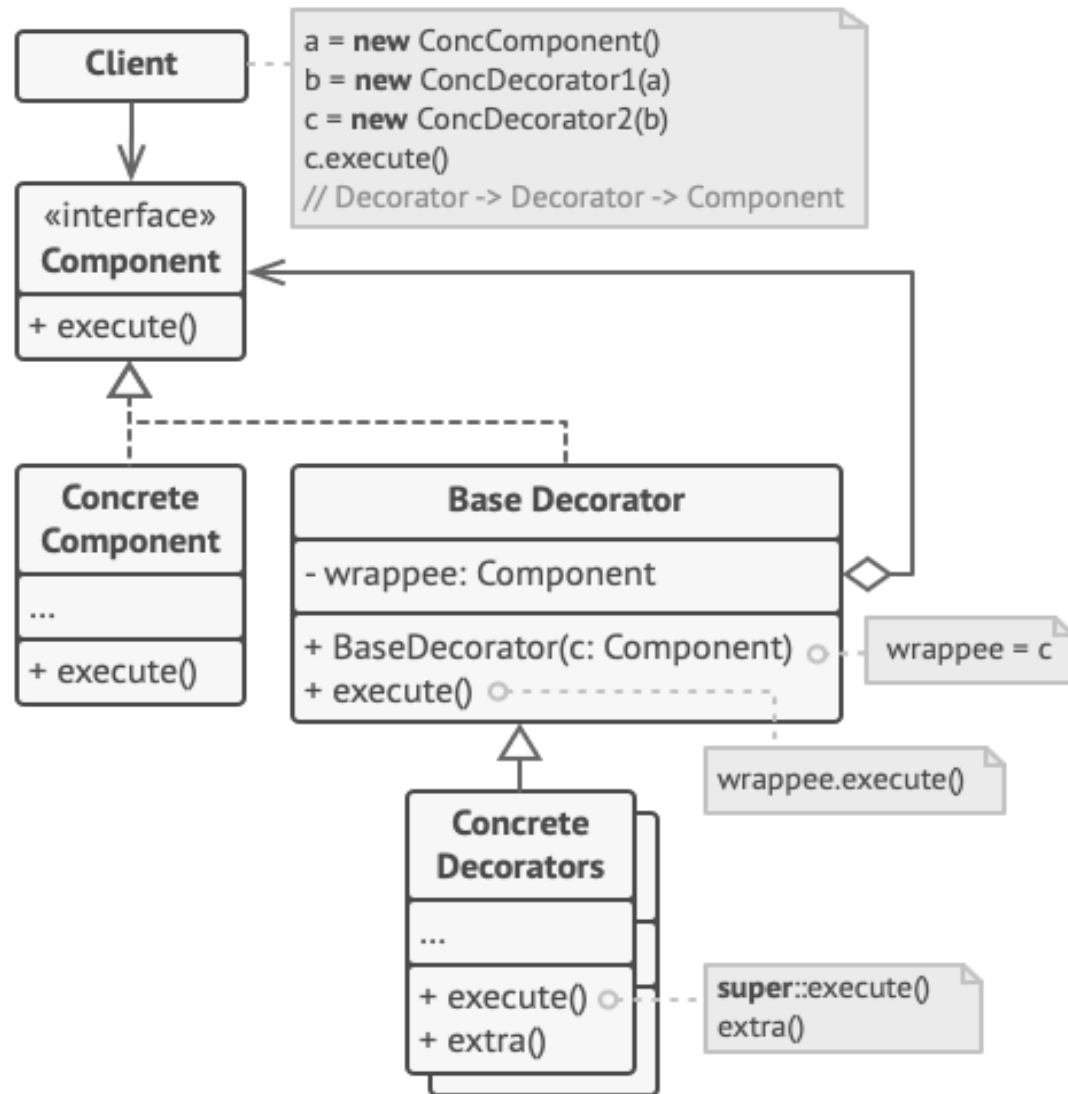
 Decorator Pattern:

Cho phép thêm hành vi vào một đối tượng riêng lẻ một cách động (dynamically) mà không ảnh hưởng đến các đối tượng khác cùng lớp.

 Ý tưởng cốt lõi:

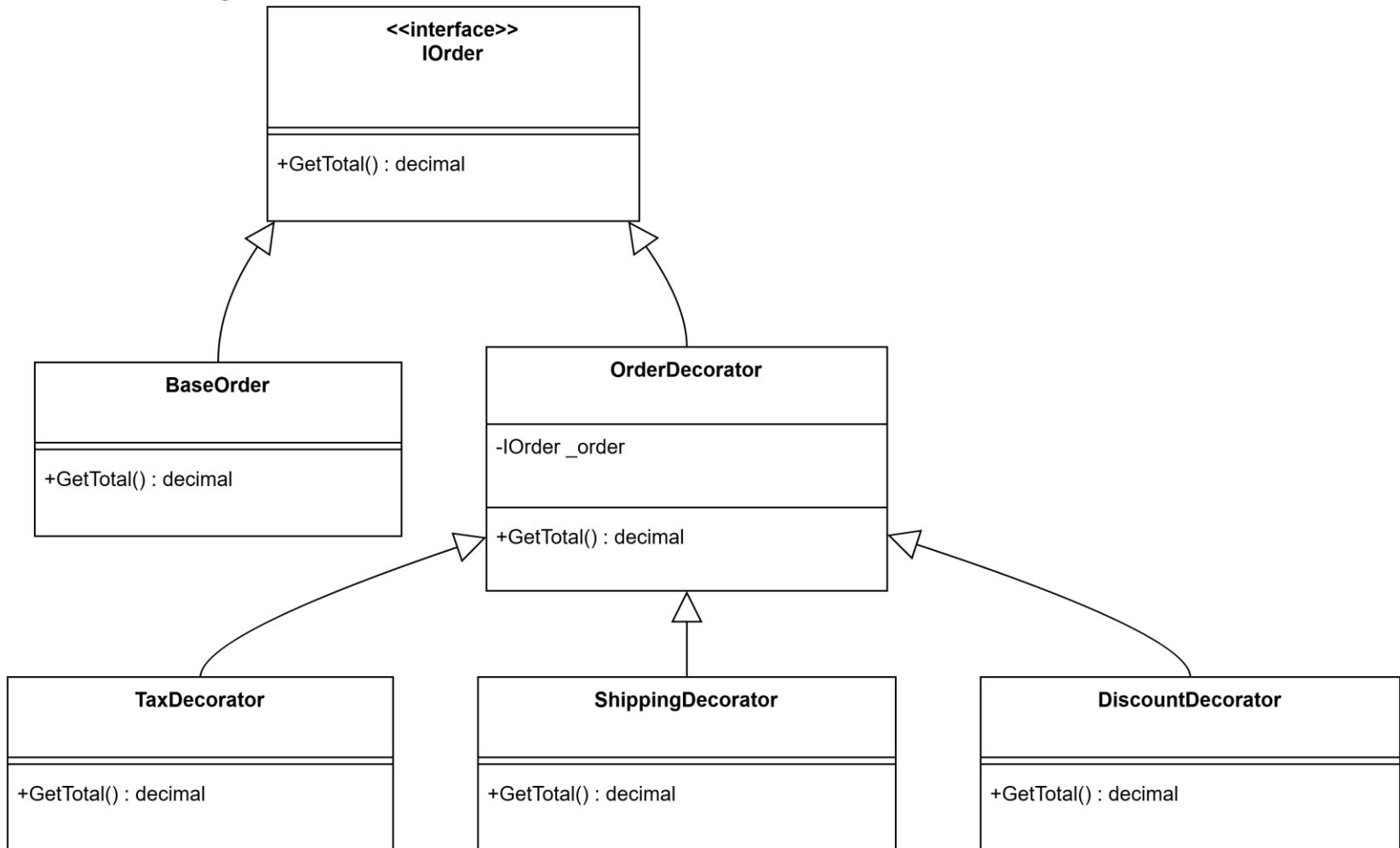
- Sử dụng Composition thay vì Inheritance.
- "Gói" đối tượng gốc trong một lớp vỏ (wrapper).
- Lớp vỏ có cùng Interface với đối tượng gốc.

4 Structure of Decorator Pattern



4 Decorator Pattern: Case study

- Component (IOrder) định nghĩa interface chung. Decorator giữ tham chiếu đến Component.



5 Triển khai: Component cơ sở

1. Component Interface

```
public interface IOrder {  
    double GetTotalCost();  
    string GetDescription();  
}
```

2. Concrete Component (Đơn hàng gốc)

```
public class RegularOrder : IOrder {  
    public double GetTotalCost() => 100.0; // Giá gốc  
    public string GetDescription() => "Regular  
Order";  
}
```

6 Triển khai: Base Decorator

Decorator triển khai IOrder và chứa 1 instance của IOrder.

3. Base Decorator

```
public abstract class OrderDecorator : IOrder {  
    protected IOrder _order; // Reference to wrapped object  
  
    public OrderDecorator(IOrder order) {  
        _order = order;  
    }  
  
    public virtual double GetTotalCost() => _order.GetTotalCost();  
    public virtual string GetDescription() =>  
_order.GetDescription();  
}
```


7 Triển khai: Các tính năng cộng thêm

4. Concrete Decorators

```
public class TaxDecorator : OrderDecorator {  
    public TaxDecorator(IOrder order) : base(order) { }  
    public override double GetTotalCost() => base.GetTotalCost() * 1.1;  
    public override string GetDescription() => base.GetDescription() + " + Tax";  
}  
  
public class ShippingDecorator : OrderDecorator {  
    public ShippingDecorator(IOrder order) : base(order) { }  
    public override double GetTotalCost() => base.GetTotalCost() + 20.0;  
    public override string GetDescription() => base.GetDescription() + " + Ship";  
}
```

8 Sử dụng (Client Code)



Linh hoạt: Có thể wrap bao nhiêu lớp tùy thích theo thứ tự.

```
// Client Code
IOrder myOrder = new RegularOrder();    // Cost: 100
Console.WriteLine(myOrder.GetTotalCost());
```

```
// Gói thêm thuế
myOrder = new TaxDecorator(myOrder);    // Cost: 110
Console.WriteLine(myOrder.GetTotalCost());
```

```
// Gói thêm ship (trên order đã có thuế)
myOrder = new ShippingDecorator(myOrder); // Cost: 110 + 20 = 130
Console.WriteLine(myOrder.GetDescription());
// Output: "Regular Order + Tax + Ship"
```

9 So sánh: Inheritance vs Decorator

Tiêu chí	Inheritance (Kế thừa)	Decorator
Mở rộng tính năng	Tĩnh (Compile-time)	Động (Runtime)
Kết hợp tính năng	Tạo class mới cho mỗi tổ hợp	Mix & Match thoải mái
Số lượng class	Bùng nổ (Explosion)	Ít, chỉ viết class chức năng
SRP (SOLID)	Class ôm nhiều việc	Mỗi Decorator 1 nhiệm vụ

10 Liên hệ thực tế

🌐 Ứng dụng thực tế trong .NET:

1. I/O Streams:

FileStream -> BufferedStream -> GZipStream
(Gói các luồng xử lý dữ liệu)

2. ASP.NET Core Middleware:

Request -> Logger -> Auth -> MVC -> Response
(Một dạng pipeline giống chuỗi Decorator)

3. Logging:

Logger -> FileLogger -> EmailLogger

1 1 Bài tập

 Bài tập thực hành:

Bài 1 (Cơ bản):

Tạo hệ thống Coffee:

Espresso (30k) -> Add Milk (+5k) -> Add Mocha (+10k) -> Add Whip (+5k).

Tính tổng tiền.

Bài 2 (Nâng cao - Order System):

Áp dụng DiscountDecorator:

- Nếu là VIP Member: Giảm 10% trên tổng bill.
- Lưu ý thứ tự: Tính Discount trước hay sau Tax?

Bài 3 (Suy ngẫm):

Tại sao Middleware gọi là "Chain of Responsibility" lại với "Decorator"?

1 2 Tổng kết

“ Decorator giống như việc mặc quần áo.

Bạn có thể mặc áo sơ mi, khoác thêm áo len, rồi mặc áo mưa.

Con người bạn (Core Component) không đổi, nhưng tính năng (ấm hơn, chống nước) được thêm vào. ”

👉 Từ khóa: Wrapper, Composition > Inheritance, Runtime.